

```
struct Nodo{  
    int dato;  
    struct Nodo *siguiente;  
}  
typedef Nodo;
```

//define la unidad básica de la lista enlazada, compuesta por un dato entero y un puntero al siguiente nodo, permitiendo la construcción dinámica de la estructura//

```
typedef Nodo *PunteroNodo;
```

//define un alias de tipo para un puntero a nodo, simplificando la manipulación de la lista enlazada y mejorando la claridad del código en la interfaz del TDA//

```
void crearLista(PunteroNodo *punteroLista){  
    *punteroLista = NULL;  
}
```

//inicializa la estructura de la lista, asegurando que comience vacía y preparada para futuras operaciones//

//Entrada:

- Recibe como parámetro un puntero a un puntero de nodo (PunteroNodo *punteroLista).
- Este parámetro representa la referencia a la estructura de datos abstracta "Lista", que se implementa como una secuencia de nodos enlazados.

Salida:

- No devuelve ningún valor (tipo void).
- El puntero pasado por referencia queda apuntando a NULL.//

```
bool esVacia(PunteroNodo lista){  
    if(lista == NULL){  
        return true;  
    }else{  
        return false;  
    }  
}
```

//permite verificar si la lista enlazada está vacía, devolviendo true si no tiene nodos y false en caso contrario//

//Entrada:

- Recibe como parámetro un puntero a nodo (PunteroNodo lista).
- Este parámetro representa el inicio de la lista enlazada dentro del TDA.

Salida:

- Devuelve un valor booleano (true o false).
- true indica que la lista está vacía.
- false indica que la lista contiene al menos un elemento.//

```
void NuevoNodo(PunteroNodo *PunteroLista){
```

```

*PunteroLista = (PunteroNodo)malloc(sizeof(Nodo));
if(esVacia(*PunteroLista)){
    printf("La lista esta vacia");
    exit(1);
}
}

```

//reserva memoria para un nuevo nodo de la lista; si la reserva falla, informa el error y finaliza el programa//

//Entrada:

- Recibe como parámetro un puntero a un puntero de nodo (PunteroNodo *PunteroLista)

Descripción del proceso:

- Se solicita memoria dinámica con malloc para un nuevo nodo del tipo Nodo.
- El puntero de lista se actualiza para apuntar a esa nueva región de memoria.
- Se verifica si la asignación fue exitosa usando la función esVacia.
 - Si malloc devolvió NULL, significa que no se pudo reservar memoria.
 - En ese caso, se imprime un mensaje de error y se termina la ejecución del programa con exit(1).

Salida:

- No devuelve ningún valor (tipo void).
- El puntero pasado por referencia queda apuntando a un nuevo nodo válido, o el programa finaliza si no hay memoria disponible.

bool existeEnLista(PunteroNodo lista, int valor)

//Recorre la lista enlazada para verificar si un valor está presente; devuelve true si lo encuentra y false si no//

//Entrada:

- Recibe un puntero a nodo (PunteroNodo lista), que representa el inicio de la lista enlazada.
- Recibe un entero (int valor), que es el dato que se desea verificar si está presente en la lista.

Descripción del proceso:

- Recorre la lista nodo por nodo mientras no esté vacía.
- En cada iteración compara el campo dato del nodo actual con el valor buscado.
- Si encuentra coincidencia, retorna inmediatamente true.
- Si no coincide, avanza al siguiente nodo mediante lista = lista->siguiente.
- Si llega al final de la lista sin encontrar el valor, retorna false.

Salida:

- Devuelve un valor booleano:
 - true si el valor existe en la lista.
 - false si el valor no se encuentra en ningún nodo.

Aclaraciones:

- No modifica la lista ni sus nodos.
- Es una función de consulta que únicamente inspecciona el contenido de la lista.
- Permite determinar la pertenencia de un elemento dentro de la estructura.

void AgregarPrincipio(PunteroNodo *PunteroLista, int valorDato)

//inserta un nuevo nodo al inicio de la lista enlazada, desplazando el resto de los elementos hacia adelante//

//Entrada:

- Recibe un puntero a un puntero de nodo (PunteroNodo *PunteroLista), que representa la referencia al inicio de la lista enlazada.
- Recibe un entero (int valorDato), que es el dato que se desea insertar en la lista.

Descripción del proceso:

- Se crea un nuevo nodo mediante la función NuevoNodo.
- Se asigna al campo dato del nuevo nodo el valor recibido como parámetro.
- El campo siguiente del nuevo nodo se enlaza con el nodo que actualmente es el primero de la lista (*PunteroLista).
- Finalmente, el puntero de inicio de la lista se actualiza para que apunte al nuevo nodo, convirtiéndolo en el primer elemento.

Salida:

- No devuelve ningún valor (tipo void).
- El efecto observable es que la lista ahora tiene un nuevo nodo al inicio, con el valor indicado.

Aclaraciones:

- La lista se modifica agregando un nuevo elemento en la cabeza.
- El tamaño de la lista aumenta en uno.
- La estructura mantiene la integridad de los enlaces, ya que el nuevo nodo apunta al anterior primer nodo.//

void AgregarFinal(PunteroNodo *punteroLista, int valorDato)

//inserta un nuevo nodo al final de la lista enlazada, manteniendo el orden de los elementos ya existentes//

//Entrada:

- Recibe un puntero a un puntero de nodo (PunteroNodo *punteroLista), que representa la referencia al inicio de la lista enlazada.
- Recibe un entero (int valorDato), que es el dato que se desea insertar en la lista.

Detalles del proceso:

- Se crea un nuevo nodo mediante la función NuevoNodo.
- Se asigna al campo dato del nuevo nodo el valor recibido como parámetro.
- Se establece su campo siguiente en NULL, ya que será el último nodo de la lista.
- Si la lista está vacía (esVacia(*punteroLista)), el nuevo nodo se convierte en el primer elemento.

- Si la lista no está vacía, se recorre con un puntero auxiliar hasta llegar al último nodo (aquel cuyo siguiente es NULL).
- Una vez encontrado, se enlaza el último nodo existente con el nuevo nodo, agregándolo al final.

Salida:

- No devuelve ningún valor (tipo void).
- La lista ahora tiene un nuevo nodo al final, con el valor indicado.

Aclaraciones:

- La lista se modifica agregando un nuevo elemento en la cola.
- El tamaño de la lista aumenta en uno.
- La estructura mantiene la integridad de los enlaces, ya que el nuevo nodo se enlaza correctamente al final.//

void AgregarOrdenado(PunteroNodo *punteroLista, int valorDato)

//inserta un nuevo nodo en la lista enlazada manteniendo el orden ascendente de los valores//

//Entrada:

- Recibe un puntero a un puntero de nodo (PunteroNodo *punteroLista), que representa la referencia al inicio de la lista enlazada.
- Recibe un entero (int valorDato), que es el dato que se desea insertar en la lista.

Detalles del proceso:

- Se crea un nuevo nodo mediante la función NuevoNodo.
- Se asigna al campo dato del nuevo nodo el valor recibido como parámetro.
- Se inicializa su campo siguiente en NULL.
- Si la lista está vacía o el primer nodo tiene un valor mayor o igual al que se quiere insertar, el nuevo nodo se coloca al inicio.
- Si no, se recorre la lista con un puntero auxiliar (actual) hasta encontrar la posición correcta:
- Se avanza mientras el siguiente nodo exista y su valor sea menor que el valor a insertar.
- Al encontrar el lugar adecuado, se enlaza el nuevo nodo entre actual y actual->siguiente.

Salida:

- No devuelve ningún valor (tipo void).
- El efecto observable es que la lista ahora contiene el nuevo nodo en la posición ordenada según el valor.

Aclaraciones:

- La lista se modifica agregando un nuevo elemento en la posición correcta para mantener el orden ascendente.
- El tamaño de la lista aumenta en uno.
- La estructura conserva la integridad de los enlaces y la propiedad de orden.

void eliminarElemento(PunteroNodo *punteroLista, int valorDato)

// busca y elimina el nodo que contiene un valor específico en la lista enlazada, ajustando los punteros y liberando la memoria asociada//

//Entrada:

Recibe un puntero a un puntero de nodo (PunteroNodo *punteroLista), que representa la referencia al inicio de la lista enlazada.

Recibe un entero (int valorDato), que es el dato que se desea eliminar de la lista.

Detalles del proceso:

- Primero verifica si el valor existe en la lista mediante la función existeEnLista.
- Si el valor está presente:
 - Se inicializa un puntero auxiliar al inicio de la lista.
 - Si el primer nodo contiene el valor, se elimina directamente actualizando el puntero de inicio al siguiente nodo.
 - Si no es el primer nodo, se recorre la lista con dos punteros (anterior y auxiliar) hasta encontrar el nodo con el valor buscado.
 - Una vez encontrado, se ajusta el enlace del nodo anterior para que apunte al siguiente del nodo eliminado.
 - Finalmente, se libera la memoria del nodo eliminado con free.
- Si el valor no está en la lista, se muestra un mensaje indicando que no existe.

Salida:

- No devuelve ningún valor (tipo void).
- El nodo con el valor indicado se elimina de la lista, o se informa que no existe.

Aclaraciones:

- La lista se modifica reduciendo su tamaño en uno si el valor estaba presente.
- Se mantiene la integridad de los enlaces, evitando nodos huérfanos.
- Si el valor no estaba, la lista permanece igual.//

void imprimirLista(PunteroNodo lista)

///recorre la lista enlazada e imprime en consola los valores de cada nodo, mostrando el estado actual de la estructura//

//Entrada:

- Recibe un puntero a nodo (PunteroNodo lista), que representa el inicio de la lista enlazada.

Detalle del proceso:

- Recorre la lista nodo por nodo mientras no esté vacía.
- En cada iteración imprime el valor almacenado en el campo dato del nodo actual, seguido de un separador " | ".
- Avanza al siguiente nodo mediante lista = lista->siguiente.
- El ciclo termina cuando se llega al final de la lista (cuando el puntero es NULL).

Salida:

- No devuelve ningún valor (tipo void).
- Imprime en consola de todos los elementos de la lista en orden, separados por " | ".

Aclaraciones:

- No modifica la lista ni sus nodos.
- Es una función de visualización que permite inspeccionar el contenido actual de la lista.
- Facilita la interacción con el usuario mostrando los datos almacenados en la estructura.